

Solution Requirements

Date	19 June 2026
Team ID	
Project Name	Rising Waters: A Machine Learning Approach to Flood Prediction
Maximum Marks	4 Marks

Step 1: Functional Requirements (FR)

Functional Requirements define the specific behaviors, functions, and features that the *Rising Waters* application will execute

S.No	Requirement Category	Requirement Description	Priority (High/Medium/Low)
1	Authentication	The web application allows users to securely register, log in, and log out. Identity verification is handled via Flask-Login session management or secure OAuth2 (e.g., Google Sign-In) to protect the user's saved prediction history dashboard.	Medium
3	External interfaces	The system interfaces with the IBM Cloud hosting platform for production deployment and data serving. It connects to open-source environmental platforms (like Kaggle APIs) for initial training dataset downloads, and communicates with a client-side web browser using standard REST APIs over HTML/JavaScript	High

4	Transactions processing	Core data exchange happens when a user submits the web input form. The Flask backend captures the parameters, validates the numeric fields, processes data scaling using <code>scaler.pkl</code> , passes the scaled vector to the <code>model.pkl</code> XGBoost engine, and logs the transactions directly into the database.	High
5	Reporting	The system generates an on-screen graphical prediction summary dashboard mapping out the precise "Flood Chance" or "No Flood Chance" probability percentage. It also displays a chronological tabular report of the user's past search metrics under the Prediction History section.	High
6	Business rules, etc.	1. Data Validation Rule: Form inputs cannot accept negative or blank rainfall/visibility values. 2. ML Pipeline Logic: Predictions cannot run unless both <code>scaler.pkl</code> and <code>model.pkl</code> load successfully into the active memory layer.	High
7	Compliance to laws or regulations	The web app ensures environmental and metadata protection by adhering to general data privacy practices (such as regional information safety	Low

		guidelines). No highly restrictive personally identifiable information (PII) is captured or stored during the risk calculations.	
8	<i>Other</i>	Automated system health alerts: The Flask application will automatically run background diagnostics to check connectivity with the cloud database and verify model stability.	Low

Step 2: Non-Functional Requirements (NFR)

Non-Functional Requirements specify the operational criteria, performance parameters, and quality metrics that judge how well the system performs.

S.No	NFR Category	Requirement Description	Target Metric / Acceptance Criteria
1	Performance & Speed	The time it takes for the backend Flask application to receive raw inputs, apply preprocessing scaling, pass them to the saved XGBoost model, and render the final prediction view.	The entire model inference and page result load must display in < 1.5 seconds from the moment the form button is clicked.
2	Scalability	The ability of the web application and backend hosting layer to handle simultaneous user requests during heavy usage or simulated peak climate stress conditions.	The deployed live application on IBM Cloud must scale resources to support up to 5,000 concurrent web users without system degradation.

3	Security & Data Privacy	The strict protection of client sessions, form interaction endpoints, and operational database files against unauthorized data manipulation.	Application configurations must enforce HTTPS communication with SSL/TLS encryption for web traffic and restrict administrative API endpoints.)
4	Reliability & Availability	The guaranteed operational uptime of the live flood-prediction portal hosted on cloud servers, ensuring disaster preparedness resources remain online.	The system must achieve a baseline availability of 99.9% uptime per month , with automated server recovery failovers active.
5	Usability & Accessibility	The interface design standard of the presentation layer dashboard, ensuring the web forms are completely responsive and easy to navigate for diverse populations.	The user dashboard interface layout must score > 85 on the System Usability Scale (SUS) and scale dynamically across mobile devices.
6	<i>Other</i>	The ease of updating the machine learning code framework and deploying modular asset modifications over time.	The application architecture must remain fully containerized so that updating the <code>model.pkl</code> file requires zero downtime to the application layer.